

Steps To Use Hibernate In Any Java Application

this is the exact flow of any hibernate application. so you must put little more concentration while you are reading this post, to understand better.

Whether the java application will run in the **server** or **without server**, and the application may be desktop or stand alone, swing, awt, servlet...what ever, but the steps are common to all.

In order to work with hibernate we **don't required any server** as mandatory but we need hibernate software (.jar(s) files).

Follow The Steps:

1. Import the hibernate API, they are many more, but these 2 are more than enough...

```
import                                     org.hibernate.*;
import org.hibernate.cfg.*;
```

2. Among **Configuration**, **Mapping xml** files, first we need to load configuration xml, because once we load the configuration file, automatically mapping file will be loaded as we registered this mapping xml in the configuration file.

So to load configuration xml, we need to create object of **Configuration** class, which is given in **org.hibernate.cfg.***; and we need to call **configure()** method in that class, by passing xml configuration file name as parameter.

Eg:

```
Configuration      cf      =      new      Configuration();
cf.configure("hibernate.cfg.xml");
```

Here our **configuration** file name is your choice, but by **default** am have been given **hibernate.cfg.xml**, so once this configuration file is loaded in our java app, then we can say that hibernate environment is **started** in our program.

So once we write the line **cf.configure("hibernate.cfg.xml")**, configuration object **cf** will reads

this xml file **hibernate.cfg.xml**, actually internally cf will use DOM parsers to read the file.

Finally...

- cf will read data from **hibernate.cfg.xml**
- Stores the data in different variables
- And finally all these variables are grouped and create one **high** level hibernate object we can call as SessionFactory object.
- So Configuration class only can create this **SessionFactory** object

```
likeSessionFactory sf = cf.buildSessionFactory();
```

Actually **SessionFactory** is an interface not a class, and **SessionFactoryImpl** is the implemented class for **SessionFactory**, so we are internally creating object of **SessionFactoryImpl** class and storing in the interface reference, so this SessionFactory object **sf** contains all the data regarding the **configuration** file so we can call **sf** as heavy weight object.

3. Creating an object of session,

- **Session** is an interface and **SessionImpl** is implemented class, both are given in **org.hibernate.***;
- When ever session is opened then internally a database connection will be opened, in order to get a session or open a session we need to call **openSession()** method in **SessionFactory**, it means SessionFactory produces sessions.

```
Session session = sf.openSession();
```

sf = SessionFactory object

4. Create a logical transaction

While working with **insert**, **update**, **delete**, operations from an hibernate application onto the database then hibernate needs a logical **Transaction**, if we are selecting an object from the database then we do **not** require any logical transaction in hibernate. In order to begin a logical transaction in hibernate then we need to call a method **beginTransaction()** given by **Session** Interface.

```
Transaction tx = session.beginTransaction();
```

session is the object of Session Interface

5. Use the methods given by **Session** Interface, to move the objects from application to database and from database to application

session .save(s) - Inserting object 's' into database

session.update(s) - Updating object 's' in the database

session.load(s) - Selecting object 's' object

session.delete(s) - Deleting object 's' from database

- So finally we need to call **commit()** in Transaction, like **tx.commit()**;
- As i told earlier, when we open session a connection to the database will be created right, so we must close that **connection** as session. close().
- And finally close the SessionFactory as **sf.close()**